

Unofficial WhatsApp CRM Integration – Complete Team Documentation

Baileys Official Docs (Guide + API)

Baileys Documentation (intro & usage)

https://baileys.wiki/docs/intro/?utm_source=chatgpt.com

Baileys GitHub Repository (source + README)

Official Baileys GitHub https://github.com/WhiskeySockets/Baileys?utm_source=chatgpt.com

NPM Package (version info + install)

@whiskeysockets/baileys on NPM

<https://www.npmjs.com/package/@whiskeysockets/baileys>

1. Purpose of This Document

This document explains **how our CRM integrates with WhatsApp (Unofficial / QR-based)** and how the team should **develop, maintain, and use** this system.

This is meant for: - Backend developers - Frontend / UI developers - QA / testers - Project managers

2. What We Built (High-Level)

We have built a **WhatsApp Messaging Service** that works like email inside CRM.

Earlier: - CRM → Email (Welcome, Invoice, Proposal, Follow-ups)

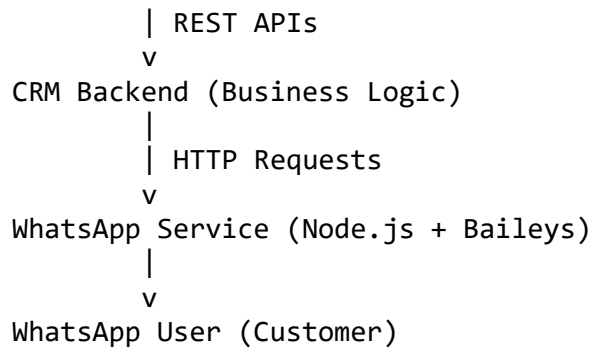
Now: - CRM → WhatsApp (Text, Link, PDF, Chat) - Optional: Email + WhatsApp together

This integration uses: - **Unofficial WhatsApp Web protocol** - **QR-based login** (no Meta / no official API)

3. System Architecture

CRM UI (Web / Admin Panel)

|



Key Design Rules

- WhatsApp runs as a **separate service (microservice)**
 - CRM never directly talks to WhatsApp
 - CRM only calls APIs
-

4. Technology Stack

WhatsApp Service

- Node.js (v18 / v20 LTS)
- Baileys library
- Express.js

CRM (Existing / Any Stack)

- PHP / Node / Laravel / Go (any)
- REST API support

Database (Recommended)

- MySQL
 - Tables: leads, chats, messages, events
-

5. Authentication & Login Flow

First-Time Login

1. Start WhatsApp service
2. Terminal shows QR code
3. Scan QR from WhatsApp → Linked Devices
4. Session saved in auth/ folder

After Login

- No QR required
- WhatsApp auto reconnects

- auth/ folder must NOT be deleted

6. CRM → WhatsApp Use Cases

CRM Event	WhatsApp Action
User signup	Welcome message
Invoice generated	Invoice link
Proposal created	Proposal PDF
Payment pending	Reminder message
Support ticket	Auto-reply
Manual follow-up	Agent chat

7. WhatsApp Service – API Specification

7.1 Send Text Message

POST /api/whatsapp/send-text

```
{
  "phone": "9198XXXXXXX",
  "message": "Welcome to our service"
}
```

7.2 Send Link (Billing / Invoice)

POST /api/whatsapp/send-link

```
{
  "phone": "9198XXXXXXX",
  "title": "Invoice Generated",
  "message": "Your invoice is ready",
  "link": "https://crm.com/invoice/INV-101"
}
```

7.3 Send File / PDF (Proposal)

POST /api/whatsapp/send-file

```
{
  "phone": "9198XXXXXXX",
  "caption": "Please find the proposal attached",
  "fileUrl": "https://crm.com/files/proposal.pdf"
}
```

7.4 Welcome Automation

POST /api/whatsapp/welcome

```
{  
  "phone": "9198XXXXXXX",  
  "name": "Customer Name"  
}
```

8. Incoming Messages (Two-Way Chat)

WhatsApp → CRM Flow

- Customer sends message
- WhatsApp service receives it
- Message saved to database
- CRM UI shows chat

Listener Logic

- Listen to `messages.upsert`
- Save sender, message, timestamp

9. CRM Chat UI Requirements

UI Layout

- Left panel: Leads list
- Right panel: Chat window

Features

- View chat history
- Send text
- Send files (PDF)
- Unread count
- Notes per lead

Looks similar to WhatsApp Web inside CRM.

10. Database Design (Recommended)

leads

- id
- name
- phone
- status
- created_at

messages

- id
 - lead_id
 - direction (in/out)
 - message
 - type (text/file/link)
 - created_at
-

11. Anti- Ban & Safety Rules (CRITICAL)

To keep WhatsApp number safe: - Do NOT bulk blast - Use event- based messaging only - Delay 3–5 seconds per message - Start with 20–30 messages/day - Use dedicated WhatsApp number

12. Deployment Guidelines

Local

- Ubuntu / WSL
- Node.js LTS

Production (Recommended)

- VPS (2 GB RAM)
 - PM2 or Docker
 - Static IP
-

13. Limitations (Must Know)

- Unofficial API (WhatsApp TOS risk)
- Number ban possible if abused
- Not recommended for bulk marketing
- Best for CRM, support, follow- ups

14. When to Move to Official API

Switch to official WhatsApp Business API when: - High volume messaging - Marketing campaigns - Long-term enterprise use

15. Summary

- WhatsApp is now a **CRM communication channel** like email
 - Integration is **event-driven**
 - System is **modular and scalable**
 - Safe usage = long-term stability
-

16. Next Phase (Future Enhancements)

- Message queue
 - Multi-agent support
 - Analytics & delivery status
 - AI auto-replies
-

17. Code-Level Documentation (Current Deployed Setup)

This section explains the **exact code structure and logic** used in the currently deployed WhatsApp CRM service.

17.1 Project Folder Structure

whatsapp-crm/

```
|  
├── server.js           # Express server + CRM APIs  
├── whatsapp.js        # WhatsApp connection & session handling  
├── auth/              # WhatsApp session files (DO NOT DELETE)  
├── node_modules/  
├── package.json  
└── package-lock.json
```

17.2 whatsapp.js – WhatsApp Connection Service

Purpose

- Handles WhatsApp login (QR-based)

- Maintains session
- Auto-reconnects on disconnect
- Exposes socket & connection status to CRM

Key Responsibilities

- Create WhatsApp socket
- Listen to connection lifecycle events
- Print QR in terminal (first login)
- Save credentials

Core Logic Flow

1. Load auth state from auth/
2. Create socket using Baileys
3. Listen to connection.update
4. Mark connection READY when open
5. Auto-reconnect if disconnected (non-logout)

Important Variables

- sock → WhatsApp socket instance
 - ready → connection readiness flag
-

17.3 server.js – CRM API Server

Purpose

- Acts as bridge between CRM and WhatsApp
- Exposes REST APIs for messaging

Server Initialization

- Creates Express app
 - Starts WhatsApp service on boot
 - Listens on port 3000
-

17.4 API: /chat (Normal CRM Link)

Endpoint

GET /chat

Example Usage

http://localhost:3000/chat?name=Dinesh&phone=9198XXXXXXX

What This API Does

1. Validates input (name, phone)
 2. Checks WhatsApp READY status
 3. Verifies phone exists on WhatsApp
 4. Adds safety delay (2 seconds)
 5. Sends presence update
 6. Sends WhatsApp text message
-

17.5 Message Send Safety Logic

To avoid WhatsApp connection issues: - READY flag is mandatory - 2-second delay before sending - Presence update before message - Error handling with retry message

17.6 Connection Lifecycle Handling

States Handled

- connecting
- open (READY)
- close (disconnect)

Auto-Reconnect Rule

- If disconnect reason $\neq$ loggedOut $\rightarrow$ auto restart
 - If loggedOut $\rightarrow$ manual QR scan required
-

17.7 auth/ Folder (Critical)

Purpose

- Stores WhatsApp session keys
- Prevents re-login every time

Rules

- DO NOT delete during normal operation
 - Delete only if:
 - WhatsApp logs out
 - Session becomes invalid
-

17.8 Logging & Debugging

Normal Logs

- connected to WA
- logging in
- CONNECTED & READY

Warning Logs (Expected)

- history sync
- buffering events

Error Logs (Action Required)

- Logged out (401)
 - Connection Closed repeatedly
-

17.9 Deployment Notes (Current Setup)

- Environment: Ubuntu / WSL
 - Node.js: v20 LTS
 - WhatsApp Login: QR-based
 - Usage Type: Low-volume CRM messaging
-

17.10 Known Limitations

- Unofficial WhatsApp protocol
 - No delivery receipts guarantee
 - Risk if bulk messaging attempted
-

17.11 Recommended Next Enhancements

- Incoming message listener
 - Message queue (rate control)
 - MySQL integration
 - CRM chat UI
-

Code-Level Documentation

1`whatsapp.js` (FULL FILE – PRODUCTION SAFE)

```
const {
```

```
  default: makeWASocket,
```

```
useMultiFileAuthState,  
DisconnectReason  
} = require("@whiskeysockets/baileys");  
  
const qrcode = require("qrcode");  
  
let sock;  
let ready = false;  
  
async function startWhatsApp() {  
  const { state, saveCreds } = await useMultiFileAuthState("auth");  
  
  sock = makeWASocket({  
    auth: state,  
    connectTimeoutMs: 60_000,  
    keepAliveIntervalMs: 10_000  
  });  
  
  sock.ev.on("creds.update", saveCreds);  
  
  sock.ev.on("connection.update", async (update) => {  
    const { connection, lastDisconnect, qr } = update;  
  
    // QR only first time  
    if (qr) {  
      console.log("❏ Scan QR from WhatsApp > Linked Devices");  
      console.log(await qrcode.toString(qr, { type: "terminal" }));  
    }  
  })  
}
```

```

if (connection === "open") {
  ready = true;
  console.log("☐ WhatsApp CONNECTED & READY");
}

if (connection === "close") {
  ready = false;
  const reason = lastDisconnect?.error?.output?.statusCode;
  console.log("☐ WhatsApp disconnected, reason:", reason);

  if (reason !== DisconnectReason.loggedOut) {
    console.log("☐ Reconnecting WhatsApp...");
    startWhatsApp();
  } else {
    console.log("☐ Logged out. Delete auth folder & scan QR again.");
  }
}
});

return {
  sock,
  isReady: () => ready
};
}

module.exports = startWhatsApp;

```

2 server.js (FULL FILE – CRM API)

```
const express = require("express");
const startWhatsApp = require("./whatsapp");

const app = express();
let wa;

// start WhatsApp on server boot
(async () => {
  wa = await startWhatsApp();
})();

// CRM → WhatsApp normal link
app.get("/chat", async (req, res) => {
  const { name, phone } = req.query;

  if (!name || !phone) {
    return res.send("Invalid Link");
  }

  if (!wa || !wa.isReady()) {
    return res.send("WhatsApp not ready yet. Try again in 10 seconds.");
  }

  const jid = phone.replace(/\D/g, "") + "@s.whatsapp.net";
  const message = `Hi ${name}, thanks for contacting us ☑`;

  try {
    // verify WhatsApp number
    const [result] = await wa.sock.onWhatsApp(jid);
    if (!result?.exists) {
      return res.send("This number is not on WhatsApp");
    }

    // stability delay
    await new Promise(r => setTimeout(r, 2000));

    // presence
    await wa.sock.sendPresenceUpdate("composing", jid);

    // send message
    await wa.sock.sendMessage(jid, { text: message });

    res.send(`<h2>Message sent to ${name}</h2>`);
  } catch (err) {
    console.error("SEND ERROR:", err.message);
    res.send("Failed to send message. Please retry after a few seconds.");
  }
});

app.listen(3000, () => {
  console.log("CRM running on http://localhost:3000");
});
```

3Developer Testing Steps (Doc me paste karo)

```
npm install  
node server.js
```

Browser test:

<http://localhost:3000/chat?name=Test&phone=91XXXXXXXXXX>

Expected:

- Terminal: ☐ WhatsApp CONNECTED & READY
- Phone: WhatsApp message received